**Computing manuscript No.**
(will be inserted by the editor)

# Load Feedforward, Not Prediction: An Empirical Comparison of PI and Model Predictive Control of the MongoDB Journal Commit Interval under Load Bursts

**Imane Jerrari · Ismail Assayad**

Received: date / Accepted: date

**Abstract** Database engines expose a write-behind setting, the journal commit interval, that trades durability against write cost, yet it is almost always fixed at deployment. We ask whether a controller can keep the volume of journal data not yet synchronised to disk near a target when the write load bursts, and whether predicting the load helps. On MongoDB we derive the controlled variable from engine counters, identify the plant, and compare seven controllers in a pre-specified, interleaved, randomised design on a real cluster (135 valid runs, tuned with an equal budget). A tuned feedback-only PI controller has a burst error of 9.55 % against 1.25–2.81 % for every controller that uses the observed load. Model predictive control with persistence forecasting is equivalent to a plain load feedforward with a model-bias observer on both primary criteria; moving-average forecasting is worse and ARIMA brings no gain. The responsiveness of the load estimate is the parameter that matters, and a 50 % higher gain makes PI with feedforward oscillate. A replication on a second platform reproduces these findings but not the lower burst error of PI with feedforward. The conclusions are limited to one engine and single-node deployments, and the signal measures exposure, not observed data loss.

**Keywords** Model predictive control · PI control · Feedforward · MongoDB · Write-behind · Recovery point objective · Journal commit interval

I. Jerrari
Laboratory of Information Systems (LIS), Faculty of Sciences Ain Chock, Hassan II University of Casablanca, Morocco
E-mail: imane.jerrari-etu@etu.univh2c.ma

I. Assayad
ENSEM, Hassan II University of Casablanca, Morocco
E-mail: i.assayad@ensem.ac.ma

## 1 Introduction

Stateful workloads on container platforms must meet data-protection commitments, usually expressed as a Recovery Point Objective (RPO), the maximum tolerable window of data loss, inside a Service Level Agreement (SLA). Database engines trade this window against write cost through a write-behind setting: in MongoDB, the WiredTiger journal is flushed every `journalCommitInterval` milliseconds. The setting is static in operators and deployment manifests, so a load burst changes the amount of data exposed between two synchronisations without any reaction.

A feedback controller needs a measurable signal and a model of how the actuator moves it. Predictive control is attractive because it can anticipate disturbances [1], but its benefit over a well-designed classical controller is often assumed rather than measured. This paper measures it on a real cluster, for one engine and one knob, and takes care that the comparison is fair: the signal is derived from engine counters only, the plant is identified before any controller is designed, every controller family receives the same tuning budget, runs are interleaved and randomised, the analysis is fixed before the data are seen, and a claim of "no difference" is supported by an equivalence test rather than by a non-significant difference test.

*Contributions.* (1) A controlled variable, the journal volume exposed per synchronisation cycle, computed from two WiredTiger counters, with an identified plant: static law, sensor window and actuator latency (Section 3). (2) A fair comparison design with seven controllers, including the minimal baseline that matters most: load feedforward with a model-bias observer and neither feedback nor prediction (Sections 4–5). (3) Results on 135 valid real-cluster runs and a sensitivity analysis of 81 more: load information, not prediction or optimisation, explains the gain over a feedback-only PI controller (Section 6). (4) An explicit account of what the study does not establish (Section 8).

*Scope.* One engine (MongoDB 8.0.32, single pod), one main host (Kind on WSL2) plus a replication of the burst scenario on a second host, one knob. The results do not transfer to other engines without new identification, and no crash-recovery experiment was run: the signal measures exposure of journal data, not observed data loss.

## 2 Related work

Look-ahead and model predictive control have been applied to power and performance management of virtualised environments [17–19]. Discrete-time and PID controllers have been proposed for containerised applications and autoscaling [11, 12, 8, 9], with stability concerns for the Kubernetes autoscaler loop [10]. Forecasting-based provisioning combines a predictor with the resource decision [6, 7, 16]. Learning-based configuration tuning for databases targets

aggregate throughput or latency [13–15] rather than a bound on exposed data. Offset-free MPC uses a disturbance model to remove steady-state error [2, 3]; the bias observer used below plays that role. Integrator windup and its remedies are classical [5, 4]. We are not aware of a measured comparison of predictive and feedforward control of a database durability setting on a real cluster with an equal tuning budget.

## 3 Controlled variable and plant

### 3.1 Signal

MongoDB exposes cumulative counters of its WiredTiger journal in `serverStatus`: bytes written, flush operations and sync operations. We define the exposed volume over a window of $W = 10\,\mathrm{s}$ (20 control ticks of $0.5\,\mathrm{s}$) as

$$V = \frac{\Delta(\text{log bytes written})}{\Delta(\text{log sync operations})}, \tag{1}$$

the average volume of journal data written between two synchronisations. The signal is computed from counters only; it never reads the commanded interval $J$. It is a composite of two counters, not a direct measurement of data loss. The actuator is $J$, set with `setParameter`; the disturbance is the byte rate $R$.

### 3.2 Identification

*Static law.* With a controller disconnected, we imposed $J \in \{20, 100, 500\}\,\mathrm{ms}$ at 100, 400 and 1600 documents/s (60 s per cell). Outside a floor set by one write batch ($b_0 \approx 513\,\mathrm{B}$), the data follow

$$V \approx \frac{R\,(J + pL_0)}{p}, \quad p = \min\!\left(1, \frac{RJ}{b_0}\right), \tag{2}$$

where $p$ is the fraction of flushes that trigger a sync and $L_0 \approx 7\,\mathrm{ms}$ the mean sync latency. A log-linear fit on the six points above the floor gives $\ln V = -6.68 + 1.001 \ln R + 0.959 \ln J$ (maximum error 1.4 %). In logarithms the actuator acts additively with a gain close to one and the load is an additive disturbance, which justifies the additive model used by the controllers. Table 1 and Fig. 1 compare the model with the measurements (median error 3.4 %, maximum 27.8 %). The largest errors are at $J = 20\,\mathrm{ms}$, where the measured mean sync latency (2–3 ms at 400 documents/s and above) is below the constant $L_0 = 7\,\mathrm{ms}$ of the model; the bias observer of Section 4 absorbs this mismatch.

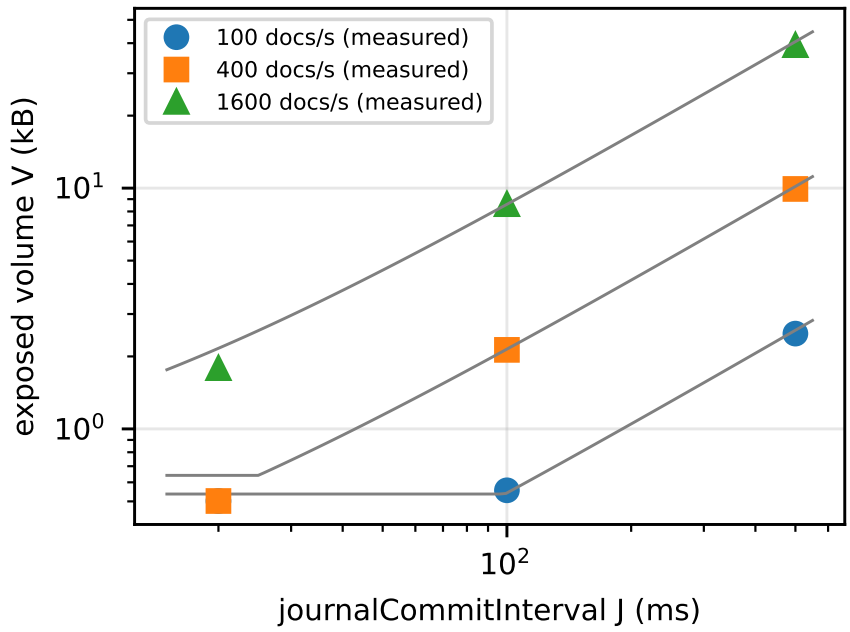



**Fig. 1** Static plant: measured exposed volume versus $J$ at three loads, with the model (grey).

**Table 1** Static plant: measured exposed volume $V$ (kB per sync) versus the model $V = R(J + pL_0)/p$, $p = \min(1, RJ/b_0)$, $b_0 = 513\,\mathrm{B}$, $L_0 = 7\,\mathrm{ms}$.

| Load (docs/s) | $J$ (ms) | $R$ (kB/s) | $V$ meas. | $V$ model | err. (%) |
|---|---|---|---|---|---|
| 100 | 500 | 5.1 | 2.49 | 2.57 | +3.4 |
| 100 | 100 | 5.1 | 0.56 | 0.54 | -2.3 |
| 100 | 20 | 5.0 | 0.50 | 0.54 | +6.9 |
| 400 | 500 | 20.0 | 9.92 | 10.16 | +2.4 |
| 400 | 100 | 20.0 | 2.13 | 2.14 | +0.7 |
| 400 | 20 | 20.0 | 0.50 | 0.64 | +27.8 |
| 1600 | 500 | 80.1 | 39.24 | 40.60 | +3.5 |
| 1600 | 100 | 80.0 | 8.58 | 8.57 | -0.1 |
| 1600 | 20 | 80.0 | 1.79 | 2.16 | +20.9 |

*Dynamics.* A load step from 400 to 1600 documents/s with $J$ fixed multiplied $V$ by 4.05 (at $J = 100\,\mathrm{ms}$) and 4.00 ($J = 200\,\mathrm{ms}$) for a load ratio of 3.98. The 10 s window makes the sensor the dominant dynamics: $V$ reaches 63 % of its final value after 6.5–7 s and 95 % after 10 s, as expected for a moving average, whereas the byte rate $R$ estimated tick by tick reaches 90 % after 2 s. The

**Table 2** Measurement noise of the 10 s signal $V$ (relative standard deviation over a 60 s segment, open loop) and syncs per window.

| Load (docs/s) | $J$ (ms) | sd($V$)/$V$ (%) | syncs per 10 s |
|---|---|---|---|
| 100 | 500 | 5.4 | 20 |
| 100 | 100 | 4.1 | 91 |
| 100 | 20 | 0.6 | 101 |
| 400 | 500 | 5.8 | 20 |
| 400 | 100 | 1.1 | 94 |
| 400 | 20 | 0.3 | 399 |
| 1600 | 500 | 5.8 | 20 |
| 1600 | 100 | 1.3 | 93 |
| 1600 | 20 | 0.4 | 448 |

actuator (`setParameter` plus read-back) takes 4.9 ms on average (maximum 6.4 ms), far below the 0.5 s control period.

*Measurement noise.* The window average is noisy mainly because of counting: at $J = 500$ ms a 10 s window contains about 20 syncs and one sync more or less changes $V$ by 5 %. The relative standard deviation of $V$ is 5.4–5.8 % at $J = 500$ ms, 1.1–4.1 % at 100 ms and 0.3–0.6 % at 20 ms (Table 2). The per-tick byte rate, used as load estimate, has a relative standard deviation of 2.5–3.6 % at 400 documents/s and 1.6–1.7 % at 1600, with a lag-one autocorrelation of -0.13 to -0.00 (base) and -0.42 to -0.32 (burst): close to white, slightly negative because of tick-boundary counting. The noise is therefore a quantisation effect, not a drifting error, and the steady-state error floor of 1.2–1.5 % seen in closed loop is of the same order.

## 4 Controllers and tuning

All controllers work on $e = \ln V^* - \ln \hat{V}$ and command $J \in [20, 500]$ ms every 0.5 s (Fig. 2).

*PI.* $u = u_0 + K_p e + K_i \sigma$ with integrator $\sigma$, clamp or back-calculation anti-windup, and optional filtering of $\ln \hat{V}$.

*Feedforward with observer (FF+obs).* With $\hat{R}$ the smoothed per-tick byte rate, $J_{\text{ff}} = V^* e^{-b}/\hat{R} - L_0$, where the bias $b$ is an exponential average of $\ln V_{\text{measured}} - \ln V_{\text{model}}$ computed over the same 10 s window; it absorbs model error (floor, $p$, $L_0$) and compensates the sensor window. There is no feedback gain and no prediction.

*PI with feedforward (PI-FF).* The same feedforward plus the PI correction; the tuner selected no bias observer for this family (bias_$\alpha = 0$) and unsmoothed load input.

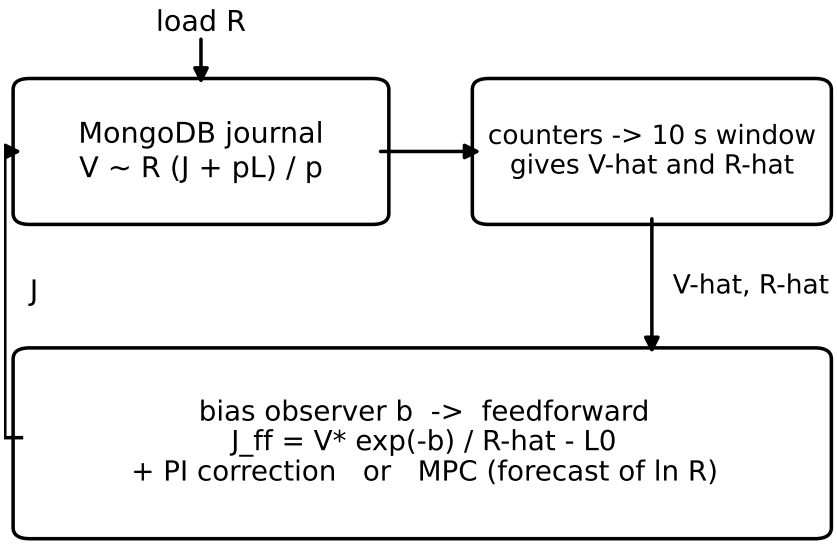



**Fig. 2** Control structure shared by all controllers. FF+obs has no PI/MPC block; PI has no feedforward; the MPC replaces the PI correction by an optimisation over a forecast of the load.

*MPC.* At each tick, $u = \ln J$ minimises $\sum_{i=1}^{H} \gamma^{i-1} s(e_i) e_i^2 + \lambda(u - u_{\text{prev}})^2$ with $e_i = \ln V^* - [\ln v(\hat{R}_{k+i}, e^u) + b]$, $v$ the static law and $s = w$ when $V$ is below the target and 1 otherwise. $\hat{R}_{k+i}$ is forecast by persistence (exponentially smoothed level), moving average (level plus least-squares slope) or ARIMA(1,1,0) fitted by least squares on the window. The scalar problem is solved by grid search and golden-section refinement: with an additive linear model the cost is convex and a line search suffices, but with the static law $v$ convexity is not guaranteed, hence the continuous refinement of a dense grid. With a flat forecast, $H$ and $\lambda$ are redundant ($He^2 + \lambda\Delta^2 \equiv e^2 + (\lambda/H)\Delta^2$) and MPC with $H = 1$ reduces to FF+obs (checked numerically).

*Tuning protocol.* Each family (PI, PI-FF, FF+obs, and each MPC predictor) received the same budget of 60 random candidates, evaluated on a calibrated simulator with a robust objective (half nominal plant, half stress plant with slower sync, more jitter and one tick of actuator delay), on tuning seeds disjoint from test seeds. The three MPC predictors share the same candidates. Search ranges were widened after a first run placed continuous optima on an edge; the edge values that remain are the "no filtering" options (value 1) and extremes of discrete sets, listed with the tuned specifications in the repository. A first tuning gave the MPC an apparent advantage over PI-FF, traced to a different smoothing of the load estimate; all controllers now receive the same raw inputs

**Table 3** Controller parameters used in all runs (random search, 60 candidates per family, robust objective; nominal values from the repository file `tuned_specs.json`). PI-default is the naive reference (no tuning). Dashes: parameter not used by the family.

| Controller | $K_p$ | $K_i$ | anti-windup | $\alpha_V$ | feedforward | $\alpha_R$ | bias $\alpha_b$ | $H$ | $\gamma$ | $\lambda$ | $w$ (below target) |
|---|---|---|---|---|---|---|---|---|---|---|---|
| PI-default | 0.3 | 0.05 | clamp | 1 | no | – | – | – | – | – | – |
| PI (tuned) | 3.19 | 0.35 | backcalc | 1 | no | – | – | – | – | – | – |
| PI-FF | 3.66 | 0.19 | backcalc | 1 | yes | 1 | 0 | – | – | – | – |
| FF+obs | 0.00 | 0.00 | clamp | 1 | yes | 1 | 1 | – | – | – | – |
| MPC persistence | – | – | – | – | – | 1 | 1 | 10 | 0.85 | 0.020 | 0.62 |
| MPC moving average | – | – | – | – | – | 0.3 | 1 | 3 | 0.7 | 0.024 | 0.86 |
| MPC ARIMA(1,1,0) | – | – | – | – | – | 0.3 | 1 | 3 | 0.7 | 0.024 | 0.86 |

Common: $V^* = 4096\,\mathrm{B}$, period $0.5\,\mathrm{s}$, $J \in [20, 500]\,\mathrm{ms}$, $J_0 = 100\,\mathrm{ms}$, window $10\,\mathrm{s}$, $b_0 = 513\,\mathrm{B}$, $L_0 = 7\,\mathrm{ms}$; MA/ARIMA window 5 / 5 samples (persistence ignores it). $\alpha_V$: filter on $\ln V$; $\alpha_R$: smoothing of the load estimate; $\alpha_b$: bias observer.

and tune their own smoothing. A PI with default gains is kept as a naive reference only. Table 3 lists every parameter used.

## 5 Experimental protocol

*Platform and workload.* A Kind cluster (Kubernetes v1.27.3, one control plane and three workers) on a single Windows host under WSL2 (6 GB), MongoDB 8.0.32 in a single authenticated pod, other database workloads stopped. The generator inserts batches of 10 documents with four threads at a regulated rate, write concern $w = 1$, $j =$ false. Target $V^* = 4096\,\mathrm{B}$.

*Scenarios.* Stationary (400 documents/s); burst (400→1600 documents/s from $t = 100$ to $160\,\mathrm{s}$); low load (100→400 documents/s, where $V^*$ is unreachable at the start because $J$ would exceed 500 ms). Runs last $300\,\mathrm{s}$ with a $20\,\mathrm{s}$ warm-up.

*Design.* Controllers are run in randomised blocks (interleaved in time), $30\,\mathrm{s}$ apart. A run is invalid and repeated if a pod restarts, the acknowledged rate falls below 95 % of the offered rate, or insertions fail. A pilot of 63 runs (3 per cell) preceded a campaign of 71 runs; the pilot used the same specifications and is pooled. Cell sizes were set after the pilot from its variability (amendment recorded before the campaign).

*Pre-specified analysis.* Primary criteria on the burst scenario: mean error during the burst and exposure peak (maximum over $2\,\mathrm{s}$ windows, relative to $V^*$). Seven contrasts, Mann–Whitney tests with Holm correction over 14 tests, Cliff's $\delta$ and bootstrap CIs; equivalence (TOST on Welch 90 % CI) for the pairs where the claim is "no difference", with margins of 0.7 percentage points (burst error) and 0.10 (peak). Margins are about half the steady-state noise floor.

*Use of AI assistance.* An AI assistant (Claude, Anthropic) was used during the study to help design the protocol and the analysis plan, to write and test the code of the controllers, the simulator, the run orchestration and the analysis scripts, to generate the figures and tables, to diagnose experimental problems

**Table 4** Burst scenario (400→1600 documents/s for 60 s): primary criteria and secondary measures, mean (sd) over $n$ valid runs (pilot included). The single run with a journal-sync stall $\geq 5$ s (FF + observer) is excluded here; all contrasts are also reported with it (Table 5). Peak = maximum exposure over 2 s windows relative to the target $V^*$.

| Controller | $n$ | Burst error (%) | Peak ($\times V^*$) | Steady error (%) | $\lvert\Delta \ln J\rvert$ per tick |
|---|---|---|---|---|---|
| PI (default gains) | 3 | 55.51 (1.80) | 2.71 (0.03) | 35.61 (0.65) | 0.005 (0.000) |
| PI (tuned) | 6 | 9.55 (0.15) | 1.88 (0.09) | 4.85 (0.13) | 0.043 (0.001) |
| PI + feedforward | 15 | 1.25 (0.17) | 1.36 (0.05) | 1.40 (0.10) | 0.055 (0.012) |
| FF + observer | 15 | 1.82 (0.30) | 1.34 (0.07) | 1.46 (0.11) | 0.032 (0.015) |
| MPC, persistence | 15 | 1.69 (0.28) | 1.36 (0.04) | 1.40 (0.12) | 0.026 (0.007) |
| MPC, moving average | 10 | 2.81 (0.45) | 1.50 (0.08) | 1.78 (0.25) | 0.024 (0.005) |
| MPC, ARIMA(1,1,0) | 10 | 1.83 (0.46) | 1.40 (0.14) | 1.59 (0.15) | 0.028 (0.009) |

and to draft and edit the text. All experiments were run by the authors on their own infrastructure; all reported data come from those runs, and every number in the tables and in the text is produced by the released scripts from the released run files.

## 6 Results

*Data.* 135 main runs (pilot 63, campaign 71, one replacement run), all valid, plus 81 sensitivity runs, all valid; no pod restarted during any run, acknowledged/offered rate between 0.964 and 1.002. The pod was recreated once between the pilot and the campaign and restarted once between two sessions of the sensitivity campaign (13 runs before, 69 after; the concurrent nominal controls were run after).

*Primary criteria.* Table 4 and Fig. 3–4 summarise the burst scenario. The tuned PI has a burst error of 9.55 % (sd 0.15) and a peak of $1.88\times V^*$, against 1.25 % (PI-FF), 1.82 % (FF+obs) and 1.69 % (MPC with persistence), with peaks of 1.36, 1.34 and 1.36. The PI-versus-PI-FF difference is +8.30 pp (95 % CI [8.17, 8.44], $p_{\mathrm{Holm}} = 4.4\times10^{-4}$).

*MPC versus feedforward with observer.* FF+obs versus MPC with persistence: +0.13 pp [-0.07, 0.32] on the burst error ($p_{\mathrm{Holm}} = 0.951$) and -0.02 on the peak. The equivalence tests (Table 6) conclude to equivalence on both criteria once the stalled run (below) is set aside, and are inconclusive with it (one extreme value widens the CI), while the rank tests do not change. MPC with a moving average has a burst error larger than persistence by 1.12 pp, and ARIMA is equivalent to persistence on the burst error; neither forecaster improves on persistence.

*PI with feedforward.* On the main platform PI-FF has the lowest burst error, 0.44 pp below MPC (Table 5), with no difference on the exposure peak; this did not replicate on the second platform (Section 6.2). It also moves $J$ about twice as much (0.055 against 0.026 in $\lvert\Delta \ln J\rvert$ per tick) and, when the target is unreachable and the actuator saturates (low-load scenario), recovers in 24.08 s against 3.00 s for MPC (Table 7).

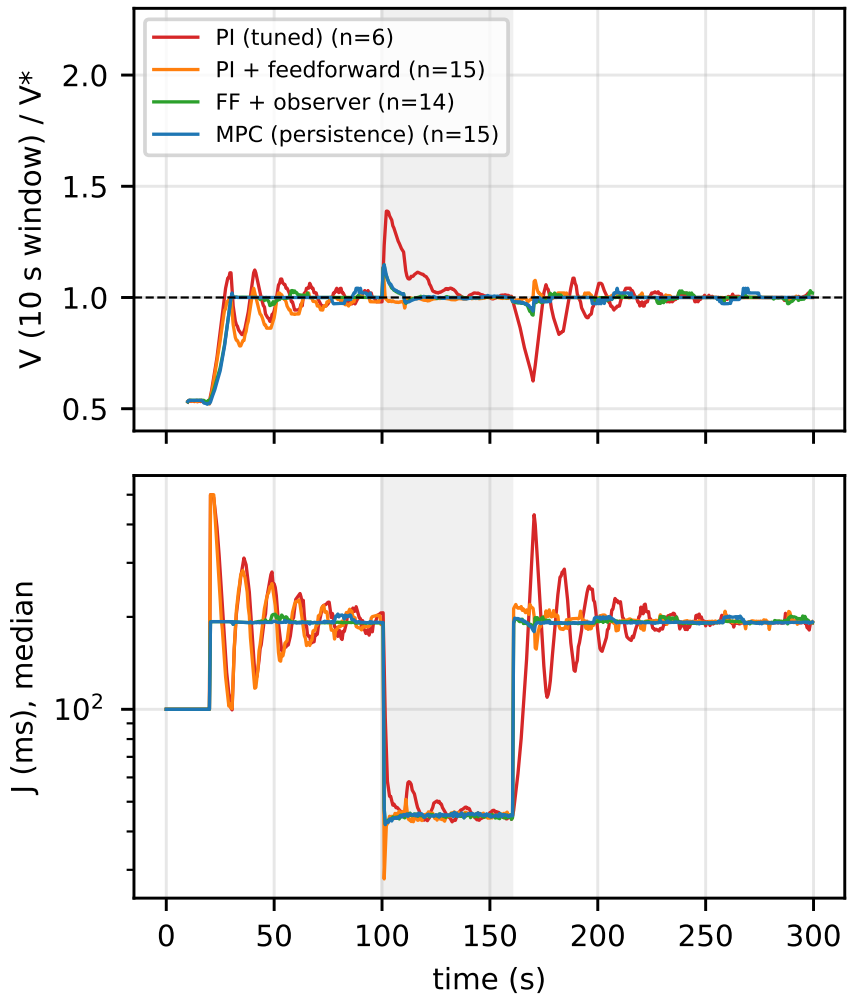



**Fig. 3** Burst scenario, median over valid runs (shaded: burst). Top: $V$ over a 10 s window relative to the target. Bottom: commanded $J$.

*A journal-sync stall.* One run (FF+obs, first run after the pod was recreated) shows no journal sync for 12.5 s during the burst, then 1.5 s and 1 s; its burst error is 171 % and its peak $39 \times V^*$. It is valid under the pre-specified rules and is included in the primary analysis; the sensitivity analysis excludes runs with a sync gap of 5 s or more (a rule defined from the counters, post hoc). The cause is unknown; two shorter gaps (1 and 1.5 s) occur in two other runs of the main analysis, and one gap of 3.5 s in a sensitivity run. Counters of journal pre-allocation are now recorded to investigate such events.

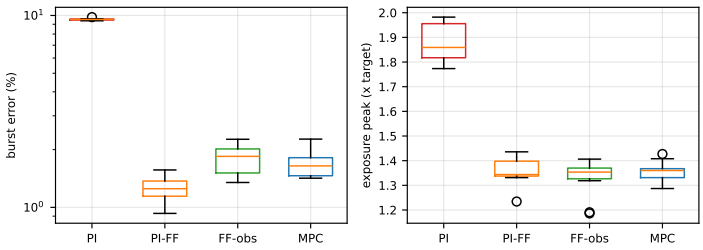



**Fig. 4** Primary criteria per controller (boxes: quartiles; points: outliers).

**Table 5** Pre-specified contrasts (Mann–Whitney U, two-sided; Holm correction over the 14 tests = 7 contrasts × 2 primary criteria). Difference = mean(A) − mean(B) with 95% bootstrap CI; $\delta$ = Cliff's delta. "All runs" includes the stalled run; "excl." removes it (post hoc sensitivity analysis).

| Contrast (A vs B) | Difference [95% CI] (all runs) | $\delta$ | $p_{\text{Holm}}$ (all) | Difference [CI] (excl.) | $p_{\text{Holm}}$ (excl.) |
|---|---|---|---|---|---|
| *Burst error (pp)* | | | | | |
| PI vs PI-FF | +8.30 [+8.17, +8.44] | +1.00 | $4.4\times10^{-4}$ | +8.30 [+8.17, +8.44] | $4.4\times10^{-4}$ |
| PI-FF vs FF+obs | -11.17 [-32.44, -0.44] | -0.93 | $1.8\times10^{-4}$ | -0.57 [-0.74, -0.40] | $2.7\times10^{-4}$ |
| FF+obs vs MPC-pers. | +10.73 [-0.02, +32.03] | +0.35 | 0.605 | +0.13 [-0.07, +0.32] | 0.951 |
| PI-FF vs MPC-pers. | -0.44 [-0.61, -0.29] | -0.92 | $2.5\times10^{-4}$ | -0.44 [-0.61, -0.29] | $2.7\times10^{-4}$ |
| MPC-pers. vs MPC-MA | -1.12 [-1.42, -0.83] | -0.95 | $9.2\times10^{-4}$ | -1.12 [-1.42, -0.83] | $9.2\times10^{-4}$ |
| MPC-pers. vs MPC-ARIMA | -0.14 [-0.43, +0.17] | -0.15 | 1.000 | -0.14 [-0.43, +0.17] | 1.000 |
| PI vs MPC-pers. | +7.86 [+7.68, +8.03] | +1.00 | $4.4\times10^{-4}$ | +7.86 [+7.68, +8.03] | $4.4\times10^{-4}$ |
| *Peak (×V*)* | | | | | |
| PI vs PI-FF | +0.52 [+0.45, +0.59] | +1.00 | 0.005 | +0.52 [+0.45, +0.59] | 0.005 |
| PI-FF vs FF+obs | -2.33 [-7.07, +0.05] | +0.10 | 1.000 | +0.02 [-0.01, +0.06] | 1.000 |
| FF+obs vs MPC-pers. | +2.34 [-0.05, +7.07] | +0.06 | 1.000 | -0.02 [-0.06, +0.01] | 1.000 |
| PI-FF vs MPC-pers. | +0.00 [-0.03, +0.03] | +0.09 | 1.000 | +0.00 [-0.03, +0.03] | 1.000 |
| MPC-pers. vs MPC-MA | -0.15 [-0.20, -0.09] | -0.81 | 0.005 | -0.15 [-0.20, -0.09] | 0.005 |
| MPC-pers. vs MPC-ARIMA | -0.04 [-0.13, +0.04] | -0.25 | 1.000 | -0.04 [-0.13, +0.04] | 1.000 |
| PI vs MPC-pers. | +0.52 [+0.46, +0.59] | +1.00 | 0.005 | +0.52 [+0.46, +0.59] | 0.005 |

**Table 6** Equivalence tests (TOST, Welch 90% CI inside ± margin; margins 0.7 pp on the burst error and 0.10 on the peak).

| Pair, criterion | 90% CI (excl.) | verdict (excl.) | verdict (all) |
|---|---|---|---|
| FF+obs vs MPC-pers., error | [-0.05, +0.31] | equivalent | inconclusive |
| MPC-pers. vs MPC-ARIMA, error | [-0.42, +0.15] | equivalent | equivalent |
| FF+obs vs MPC-pers., peak | [-0.05, +0.01] | equivalent | inconclusive |
| MPC-pers. vs MPC-ARIMA, peak | [-0.12, +0.04] | inconclusive | inconclusive |

*Simulator versus real cluster.* The simulator was calibrated on the static grid (worst error 3.9 % on $V$) and on the burst response. On the pilot the ranking of the seven controllers by burst error is identical to the simulated one (Spearman 1.00), with real-to-simulated ratios between 0.80 and 1.09. The simulator underestimates the steady-state measurement noise (real error 1.2–1.5 % versus 0.5–0.95 %) and the between-run variability, so it was used for tuning and design only.

**Table 7** Stationary (400 documents/s) and low-load (100→400 documents/s, target unreachable at the start) scenarios, mean over $n$ runs.

| Controller | Stat. err. (%) | Low-load err. (%) | Recov. (s) | $n$ |
|---|---|---|---|---|
| PI-def. | 26.08 | 49.83 | – | 3 |
| PI | 1.32 | 28.78 | 1.8 | 6 |
| PI-FF | 1.38 | 31.48 | 24.1 | 6 |
| FF+obs | 1.34 | 28.18 | 2.9 | 6 |
| MPC-pers. | 1.22 | 28.01 | 3.0 | 6 |
| MPC-MA | 1.31 | 28.22 | 2.3 | 6 |
| MPC-ARIMA | 1.49 | 28.27 | 2.8 | 6 |

### 6.1 Sensitivity to parameters and to the target

The criteria below were fixed before the runs (amendment 2). Each continuous parameter of PI, PI-FF, FF+obs and MPC (persistence) was perturbed one at a time by $\times 0.5$ and $\times 1.5$ (clipped to valid values), 3 runs per variant, with 3 concurrent nominal runs per controller (Table 8); the target was also changed (Table 9). The analysis is descriptive ($n = 3$): ranges and ratios, no tests.

*C1, parameter robustness.* Worst ratio of burst error to the concurrent nominal: PI 1.58, FF+obs 1.93, MPC 2.32, PI-FF 4.20. For FF+obs, MPC and PI-FF the large ratios come from one parameter, the smoothing of the load estimate: halving its factor multiplies the burst error by 1.93 (FF+obs), 2.32 (MPC) and 2.16 (PI-FF). The MPC weights ($\lambda$, $w$) and the observer factor change the error by $-10$ to $+16$ %. PI-FF is the only controller that loses stability: with $K_p \times 1.5$ ($K_p = 5.5$) it oscillates, with a burst error of 4.76 % and a peak of $7.45 \times V^*$.

*C2, robustness of the ranking.* On the burst error the worst load-aware variant (4.76 %) is below the best PI variant (8.07 %): the ranking survives. On the exposure peak it survives for FF+obs (largest peak 1.54) and for MPC (1.61) against the best PI variant (1.77) but not for PI-FF (7.45). As specified, over the union of the three load-aware families, C2 is therefore *not* satisfied on the peak; we report it as such.

*C3, target.* Journal syncs per second follow $1/V^*$ (about 17, 8.6 and 4.3 per second for $V^* = 2048$, 4096 and 8192 B, Table 9), the cost of a tighter target. No controller is uniformly best across targets; at the unreachable $V^* = 2048$ B the MPC has a larger error than FF+obs.

### 6.2 Replication on a second platform

To reduce the single-host limitation, the burst scenario was replayed for four controllers (PI, PI-FF, FF+obs, MPC with persistence), 6 runs each, on a shared

**Table 8** Sensitivity to controller parameters (burst scenario, $n = 3$ per variant, one-at-a-time, descriptive). Ratios are relative to 3 concurrent nominal runs of the same controller. Nominal: PI 9.36 %, PI-FF 1.13 %, FF+obs 1.64 %, MPC 1.60 %.

| Controller | Variant | Error (%) | Ratio | Peak ($\times V^*$) | $\lvert\Delta \ln J\rvert$ |
|---|---|---|---|---|---|
| PI | $K_i \times 0.5$ | 14.79 | 1.58 | 1.82 | 0.041 |
| PI | $K_i \times 1.5$ | 8.07 | 0.86 | 2.08 | 0.048 |
| PI | $K_p \times 0.5$ | 11.35 | 1.21 | 2.16 | 0.024 |
| PI | $K_p \times 1.5$ | 8.52 | 0.91 | 1.77 | 0.063 |
| PI-FF | $\alpha_R \times 0.5$ | 2.44 | 2.16 | 1.47 | 0.048 |
| PI-FF | $K_i \times 0.5$ | 2.98 | 2.63 | 1.32 | 0.057 |
| PI-FF | $K_i \times 1.5$ | 1.33 | 1.18 | 1.42 | 0.059 |
| PI-FF | $K_p \times 0.5$ | 1.47 | 1.30 | 1.33 | 0.043 |
| PI-FF | $K_p \times 1.5$ | 4.76 | 4.20 | 7.45 | 0.094 |
| FF+obs | $\alpha_b \times 0.5$ | 1.68 | 1.02 | 1.41 | 0.029 |
| FF+obs | $\alpha_R \times 0.5$ | 3.18 | 1.93 | 1.54 | 0.016 |
| MPC-pers. | $\alpha_b \times 0.5$ | 1.43 | 0.90 | 1.36 | 0.020 |
| MPC-pers. | $\alpha_R \times 0.5$ | 3.70 | 2.32 | 1.61 | 0.023 |
| MPC-pers. | $\lambda \times 0.5$ | 1.85 | 1.16 | 1.37 | 0.032 |
| MPC-pers. | $\lambda \times 1.5$ | 1.49 | 0.93 | 1.33 | 0.023 |
| MPC-pers. | $w \times 0.5$ | 1.51 | 0.95 | 1.33 | 0.026 |
| MPC-pers. | $w \times 1.5$ | 1.47 | 0.92 | 1.42 | 0.023 |

**Table 9** Change of the target $V^*$ (burst, $n = 3$): error, peak and journal syncs per second. At $V^* = 2048$ B the target is unreachable during the burst ($J$ would be below 20 ms).

| Controller | $V^*$ (B) | Error (%) | Peak | syncs/s |
|---|---|---|---|---|
| PI-FF | 2048 | 2.33 | 1.34 | 17.02 |
| PI-FF | 4096 | 1.13 | 1.40 | 8.68 |
| PI-FF | 8192 | 2.32 | 1.47 | 4.41 |
| FF+obs | 2048 | 1.74 | 1.32 | 17.23 |
| FF+obs | 4096 | 1.64 | 1.39 | 8.62 |
| FF+obs | 8192 | 2.83 | 1.40 | 4.31 |
| MPC-pers. | 2048 | 4.90 | 1.57 | 16.76 |
| MPC-pers. | 4096 | 1.60 | 1.37 | 8.61 |
| MPC-pers. | 8192 | 3.68 | 1.39 | 4.28 |

GitHub-hosted Linux runner (AMD EPYC 7763 64-Core Processor, 4 vCPU, 16 GB, kernel 7.0.0-1017-azure) with MongoDB mongo:8.0.32 in plain Docker, without WSL2 or Kubernetes. The specifications were not re-tuned. MongoDB 8.x refuses to start on this kernel (SERVER-121912, an allocator incompatibility); the community workaround `GLIBC_TUNABLES=glibc.pthread.rseq=1` was applied. It concerns the memory allocator, not the journal path, but its effect on the measurements is not verified. Three criteria were fixed before the runs: R1 (the PI burst error is at least three times that of each load-aware controller), R2 (FF+obs and MPC equivalent, same margins as above) and R3 (the PI exposure peak is the highest). All 24 runs were valid. Table 10 gives the

**Table 10** Replication on a second platform (shared Linux runner, MongoDB in plain Docker), burst scenario, mean (sd) over 6 runs; last column: same controller on the main platform.

| Controller | Error (%) | Peak ($\times V^*$) | $\lvert\Delta \ln J\rvert$ | syncs/s | Main platform (%) |
|---|---|---|---|---|---|
| PI (tuned) | 9.36 (0.27) | 1.86 (0.11) | 0.044 | 8.54 | 9.55 |
| PI + feedforward | 1.52 (0.06) | 1.38 (0.06) | 0.046 | 8.70 | 1.25 |
| FF + observer | 1.45 (0.32) | 1.37 (0.03) | 0.022 | 8.63 | 1.82 |
| MPC, persistence | 1.33 (0.12) | 1.37 (0.03) | 0.019 | 8.63 | 1.69 |

results. R1 is satisfied (ratios 6.2, 6.4 and 7.1), R2 is satisfied (error difference +0.13 pp, 90 % CI [-0.14, +0.39]; peak difference +0.00, [-0.03, +0.03]) and R3 is satisfied. One finding did not replicate: on the main platform PI-FF had a lower burst error than MPC (−0.44 pp); on the second platform the difference is +0.20 pp (95 % CI [+0.10, +0.30]), so the sign of the difference is the opposite of the main platform: it is platform-dependent and should not be generalised. The second platform widens the range of hosts, not the topology: it is still a single node.

## 7 Discussion

*Why prediction does not help here.* The actuator acts on $V$ within one control tick, while the sensor is delayed by the 10 s window. A controller that knows $\hat{R}$ and the static law can therefore set $J$ almost exactly; what remains is to correct the sensor lag and model error, which the bias observer does. Forecasting $R$ several ticks ahead adds noise, not information, because the plant has no delay to compensate: with a moving average the burst error is worse. This is consistent with the equivalence of MPC and FF+obs.

*Why feedback alone fails.* Feedback on the windowed $V$ reacts only as the window fills. Even with a high proportional gain ($K_p = 3.2$, $K_i = 0.35$ after tuning) the PI shows a burst error five to eight times larger than PI-FF, FF+obs and MPC with persistence, and a peak 1.4 times higher (Fig. 3); its response oscillates around the target.

*What matters.* The controller structure matters less than the responsiveness of the load estimate: smoothing it more than the plant requires doubles the burst error for every load-aware controller (Table 8), while the MPC weights and the observer factor are second order.

*Practical guidance.* On this engine and knob, a load feedforward with a model-bias observer is the simplest controller that attains the accuracy of MPC and it was stable over every perturbation tested. On the main platform PI with feedforward has the lowest nominal burst error (not replicated on the second platform) and the least margin: a 50 % higher gain made it oscillate, and it

moves the actuator twice as much and recovers more slowly under saturation. We therefore see feedback as optional: useful only with a gain margin that is measured, not assumed.

## 8 Limitations

(i) One engine and single-node deployments on two hosts (Kind under WSL2, and a shared Linux runner with MongoDB in Docker for the burst scenario only, which required a community workaround for a kernel incompatibility of MongoDB 8.x whose effect on the measurements is not verified); (ii) $V$ is a composite of two counters and no crash-recovery experiment verifies it as data loss; (iii) the simulator underestimates noise and between-run variability; (iv) tuning was done on the simulator and checked on a pilot; (v) one run contains a long journal-sync stall of unknown cause; (vi) the simulator reproduces the nominal behaviour but not the fragility of PI-FF near its stability limit (a 50 % higher gain does not oscillate in simulation unless one extra tick of delay is added, which reproduces it qualitatively), so unmodelled delay is a plausible but unverified cause; (vii) the pod restarted once between two sessions of the sensitivity campaign (each run individually valid); (viii) equivalence margins are set from the noise floor and a different margin could change an "inconclusive" into "equivalent" or the reverse; (ix) the sensitivity analysis is descriptive ($n = 3$ per variant) and one-at-a-time, so interactions between parameters are not explored.

## 9 Conclusion

On a real MongoDB cluster, using the observed write load is what lets a controller hold the exposed journal volume near its target during bursts. A model predictive controller with persistence forecasting is equivalent to a plain feedforward with a model-bias observer, and forecasting the load with a moving average or ARIMA does not help. The advantage of the feedforward does not depend on a precise tuning, except for the responsiveness of the load estimate, and a feedback term must be added with a measured gain margin. The ordering of PI against the load-aware controllers and the equivalence of MPC and feedforward with observer replicated on a second platform; the lower burst error of PI with feedforward did not. These conclusions are limited to the scenarios, engine and platforms tested.

## Declarations

**Funding.** No funding was received. **Competing interests.** The authors declare no competing interests. **Ethics approval.** Not applicable: the study involves no human participants or animals. **Author contributions.** All authors

contributed to the study conception and design. The experiments, data collection and analysis were performed by I. Jerrari. All authors read and approved the final manuscript. **Data availability.** The run files of the main study, of the sensitivity analysis and of the replication, the run schedules, the tuned specifications and the amendments written before each data collection are available at `https://github.com/imanejerrari-etu-bit/mpc-rpo-mongodb`, release `v1.0`. [**TODO: confirm that release v1.0 exists and add its commit hash before submission**] **Code availability.** The code (`mpcrpo`), the analysis scripts and the sources of this manuscript are in the same repository and release. **Use of AI tools.** An AI assistant (Claude, Anthropic) was used for protocol design support, code writing and testing, figure and table generation, diagnosis and drafting of the text, as documented in Section 5. The authors reviewed the outputs and take full responsibility for the content of the publication; the AI tool is not listed as an author.

## References


1. E. F. Camacho and C. Bordons. *Model Predictive Control*. Advanced Textbooks in Control and Signal Processing. Springer-Verlag London, 2nd edition, 2007.
2. K. R. Muske and T. A. Badgwell. Disturbance modeling for offset-free linear model predictive control. *Journal of Process Control*, 12(5):617–632, 2002.
3. G. Pannocchia and J. B. Rawlings. Disturbance models for offset-free model-predictive control. *AIChE Journal*, 49(2):426–437, 2003.
4. M. V. Kothare, P. J. Campo, M. Morari, and C. N. Nett. A unified framework for the study of anti-windup designs. *Automatica*, 30(12):1869–1883, 1994.
5. K. J. Åström and L. Rundqwist. Integrator windup and how to avoid it. In *Proceedings of the 1989 American Control Conference*, pp. 1693–1698. IEEE, 1989.
6. N. Roy, A. Dubey, and A. Gokhale. Efficient autoscaling in the cloud using predictive models for workload forecasting. In *Proceedings of the IEEE 4th International Conference on Cloud Computing (CLOUD'11)*, pp. 500–507, 2011.
7. Z. Gong, X. Gu, and J. Wilkes. PRESS: PRedictive Elastic ReSource Scaling for cloud systems. In *Proceedings of the International Conference on Network and Service Management (CNSM)*, pp. 9–16, 2010.
8. Z. Cai and R. Buyya. Inverse queuing model-based feedback control for elastic container provisioning of web systems in Kubernetes. *IEEE Transactions on Computers*, 71(2):337–348, 2022.
9. O. Pozdniakova, D. Mažeika, and A. Cholomskis. SLA-adaptive threshold adjustment for a Kubernetes horizontal pod autoscaler. *Electronics*, 13(7):1242, 2024.
10. B. Serracanta, A. Lukács, A. Rodriguez-Natal, A. Cabellos, and G. Rétvári. On the stability of the Kubernetes horizontal autoscaler control loop. *IEEE Access*, 13:7160–7166, 2025.
11. L. Baresi, S. Guinea, A. Leva, and G. Quattrocchi. A discrete-time feedback controller for containerized cloud applications. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2016)*, pp. 217–228, 2016.
12. M. Sabuhi, N. Mahmoudi, and H. Khazaei. Optimizing the performance of containerized cloud software systems using adaptive PID controllers. *ACM Transactions on Autonomous and Adaptive Systems*, 15(3), 2021.
13. D. Van Aken, A. Pavlo, G. J. Gordon, and B. Zhang. Automatic database management system tuning through large-scale machine learning. In *Proceedings of the 2017 ACM International Conference on Management of Data (SIGMOD'17)*, pp. 1009–1024, 2017.
14. J. Zhang, Y. Liu, K. Zhou, G. Li, Z. Xiao, B. Cheng, J. Xing, Y. Wang, T. Cheng, L. Liu, M. Ran, and Z. Li. An end-to-end automatic cloud database tuning system using deep reinforcement learning. In *Proceedings of the 2019 ACM International Conference on Management of Data (SIGMOD'19)*, pp. 415–432, 2019.

15. L. Ferreira, F. Coelho, and J. Pereira. Self-tunable DBMS replication with reinforcement learning. In *Distributed Applications and Interoperable Systems (DAIS 2020)*, LNCS vol. 12135, pp. 131–147. Springer, 2020.
16. N. S. Joshi, R. Raghuwanshi, Y. M. Agarwal, B. Annappa, and D. N. Sachin. ARIMA-PID: container auto scaling based on predictive analysis and control theory. *Multimedia Tools and Applications*, 83(9):26369–26386, 2024.
17. D. Kusic, J. O. Kephart, J. E. Hanson, N. Kandasamy, and G. Jiang. Power and performance management of virtualized computing environments via lookahead control. *Cluster Computing*, 12(1):1–15, 2009.
18. P. Padala, K.-Y. Hou, K. G. Shin, X. Zhu, M. Uysal, Z. Wang, S. Singhal, and A. Merchant. Automated control of multiple virtualized resources. In *Proceedings of the 4th ACM European Conference on Computer Systems (EuroSys'09)*, pp. 13–26, 2009.
19. Q. Fang, J. Wang, and Q. Gong. QoS-driven power management of data centers via model predictive control. *IEEE Transactions on Automation Science and Engineering*, 13(4):1557–1566, 2016.